

TEXAS A&M UNIVERSITY

ROBOTICS TEAM & LEADERSHIP EXPERIENCE

Week 6: Control Systems for Robotics

TURTLE Incubator - SO-ARM Software Stack

Agenda

Open-Loop vs. Closed-Loop Control

- Feedback systems and error signals

PID Control

- Proportional, integral, and derivative terms - what each one does

PID Tuning

- Overshoot, settling time, stability - how to read symptoms and fix them

Advanced Controllers

- Feedforward control and a brief intro to LQR

SO-ARM Application

- Write your own PID on a simulated joint, then tune the real servo PID

Demonstration + Hands-On + Deliverable

- Tune a PID in simulation and on the SO-ARM

ROS 2 + Gazebo Setup

- Install the simulation stack you will use starting next week

What You Are Building Today

By end of session you will:

- 1 Write a PID controller from scratch in Python and run it on a simulated SO-ARM joint
- 2 Log time, setpoint, actual, error, and output to a CSV file every cycle
- 3 Plot the response and diagnose oscillation, gravity droop, overshoot, and windup
- 4 Tune the real STS3215 on-chip PID by changing its P, I, and D coefficients
- 5 Run a step response test on the SO-ARM and compare gain sets on one plot
- 6 Install ROS 2 Jazzy and Gazebo so you are ready for simulation next week

This is the exact control loop running inside every servo motor, drone, and robotic arm in the world.

In simulation you can see every number. On the real servo the loop runs on the chip, so you tune it by watching the response.

Open-Loop vs. Closed-Loop

Open-Loop vs. Closed-Loop Control

Open-Loop

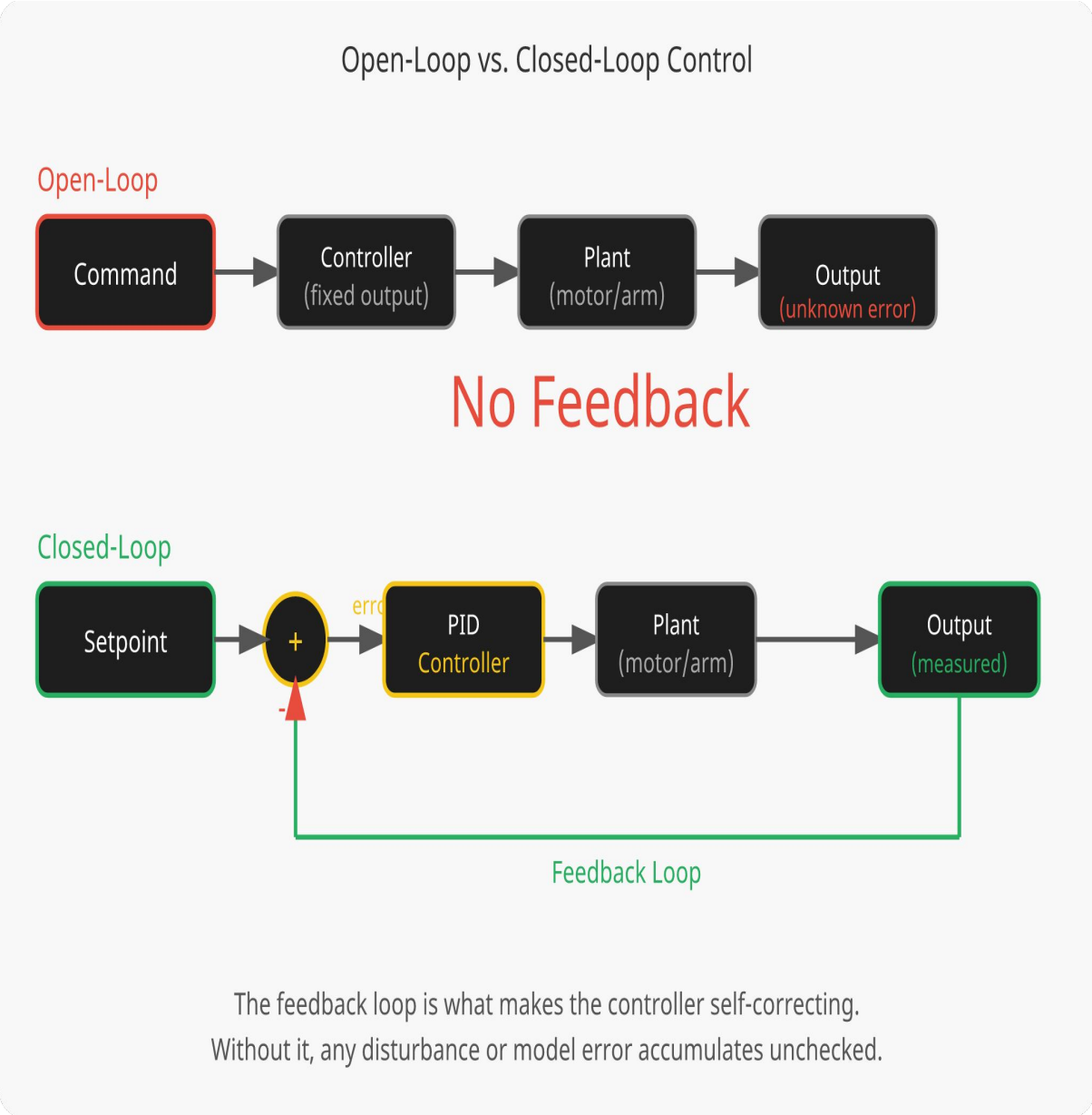
- Send a command and hope for the best
- No measurement of what actually happened
- Any disturbance (gravity, friction) goes uncorrected
- Fast and simple - used when precision doesn't matter

Closed-Loop

- Measure the output, compare to target, correct the difference
- Self-correcting - handles disturbances automatically
- Requires a sensor to close the loop
- More complex but far more accurate

The SO-ARM uses closed-loop control.

Each Feetech servo has a built-in encoder that feeds position back to its on-chip PID.



Error Signals

The error signal is the foundation of all closed-loop control:

- `error(t) = setpoint - actual_position(t)`

Positive error

- Actual is below target - controller should push harder in the positive direction

Negative error

- Actual is above target - controller should pull back

Zero error

- Arm is exactly at target - controller outputs zero (ideally)

Everything in PID is derived from this one number.

P reacts to current error. I reacts to accumulated past error. D reacts to rate of change of error.

PID Control

The Three PID Terms

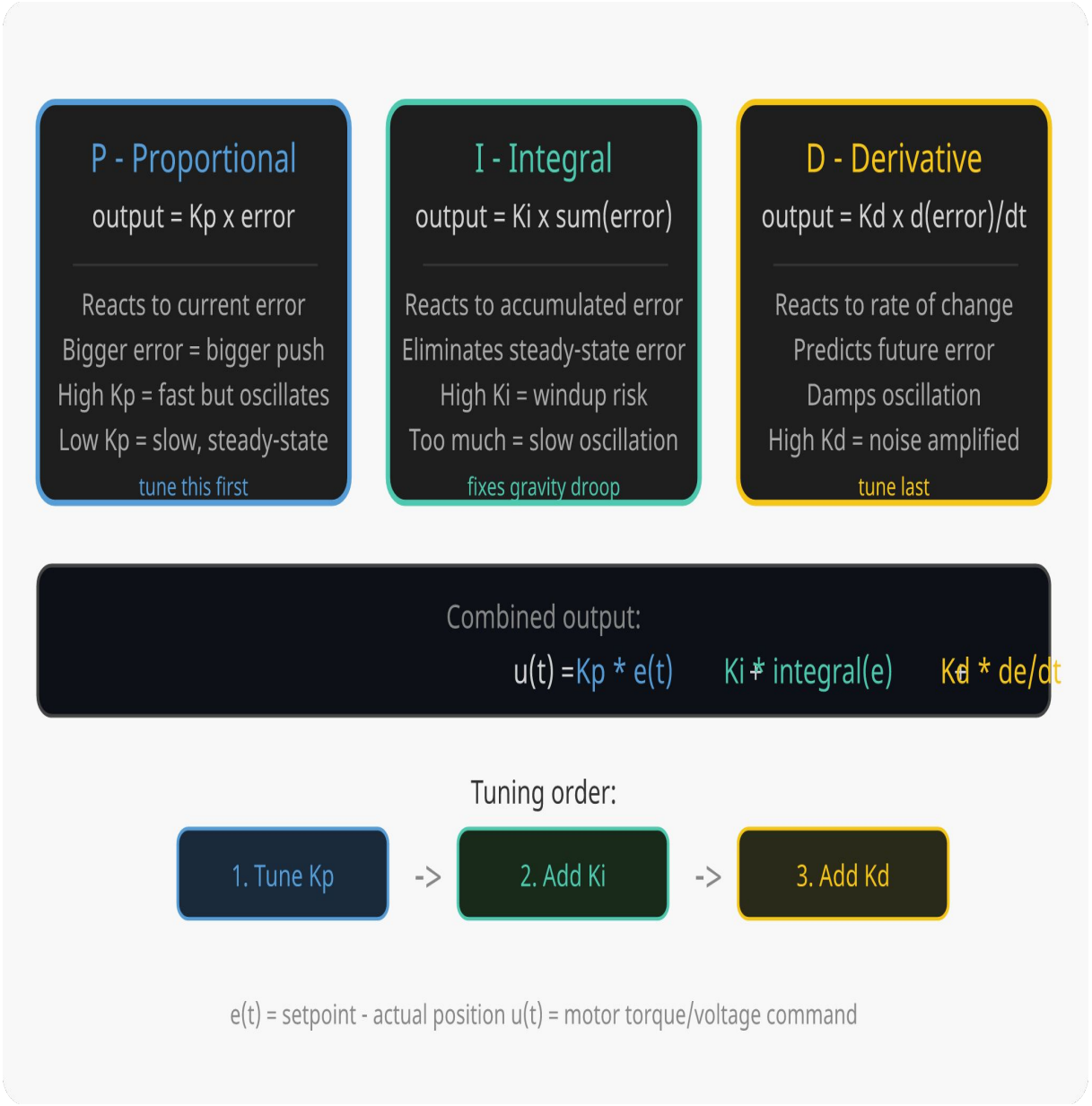
PID combines three terms:

P - Proportional
- Output proportional to current error
- Tune this first - it drives the basic response

I - Integral
- Output proportional to accumulated past error
- Eliminates steady-state error and gravity droop

D - Derivative
- Output proportional to rate of change of error
- Damps oscillation, tune last

u(t) = Kp*e + Ki*integral(e) + Kd*de/dt



PID Response Curves

What each configuration looks like:

Open Loop (red)

- Ramps up but can't self-correct to exactly 45 degrees

P Only - low gain (blue)

- Approaches target slowly, steady-state error remains

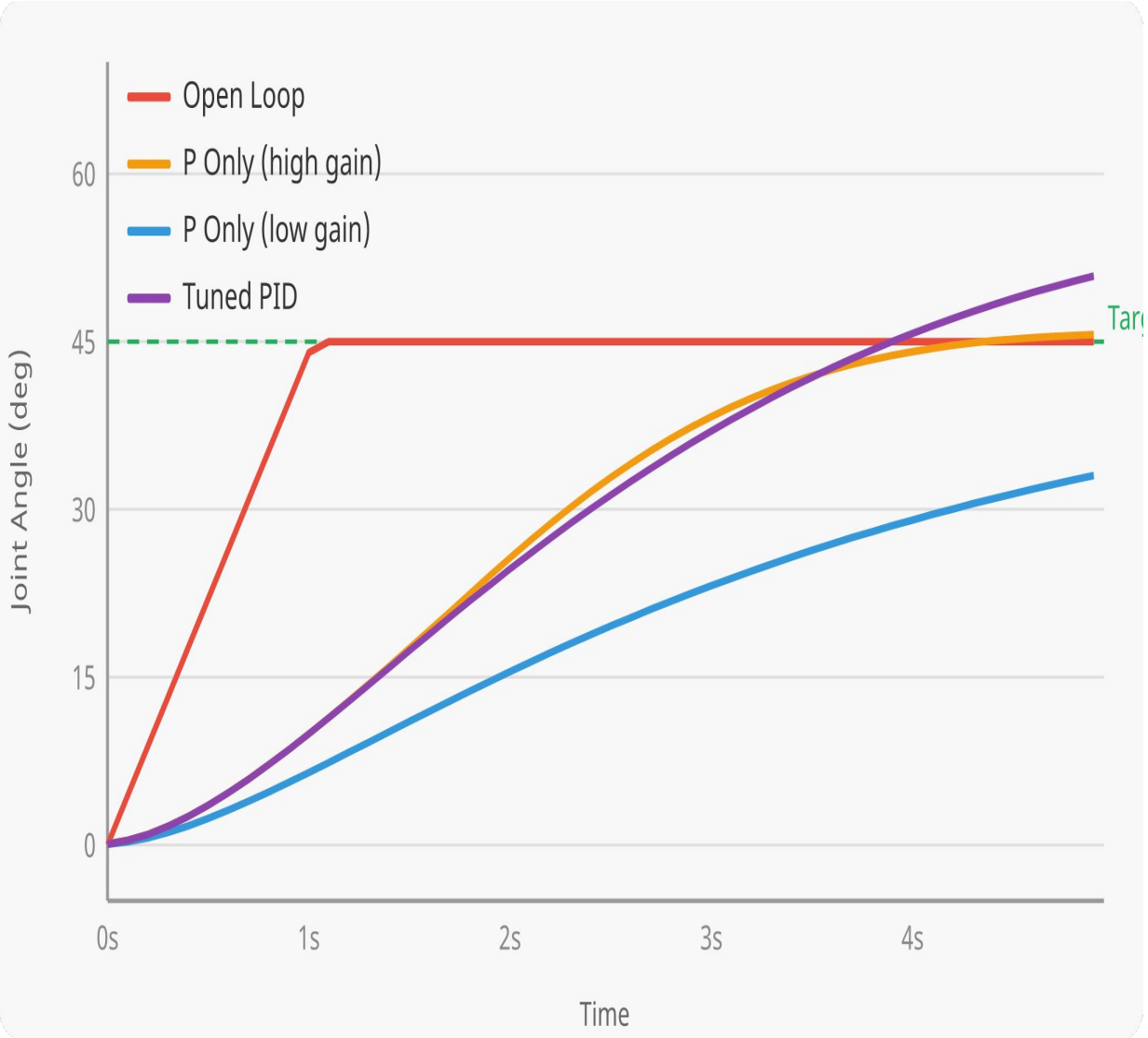
P Only - high gain (orange)

- Faster but oscillates around target

Tuned PID (purple)

- Fast, stable, settles at exactly 45 degrees

Target: 45 degrees (green dashed line)



PID Tuning

PID Tuning - Read the Symptom, Apply the Fix

Don't guess - read the symptoms:

Oscillation
- Kp too high or Kd too low

Slow response
- Kp too low

Steady-state error
- Ki too low - arm stops short of target

Overshoot
- Kp too high or Kd too low

Integral windup
- Ki too high or no clamp on integral term

PID Tuning Symptoms and Fixes

Symptom	What You See	Fix
Oscillation (hunting)	Arm bounces back and forth around setpoint endlessly	Reduce Kp Increase Kd to damp it
Slow Response (sluggish)	Arm reaches target eventually but takes too long	Increase Kp Reduce Kd if overdamped
Steady-State Error	Arm stops a few degrees short of target permanently	Increase Ki Gravity droop = more Ki
Overshoot (one-time spike)	Arm blows past target then settles back	Reduce Kp slightly Increase Kd
Integral Windup	Arm slams to limit then takes forever to recover	Clamp integral term Add anti-windup logic

Rule of thumb: start with Ki=0, Kd=0. Raise Kp until it oscillates, back off 30%, then add Ki and Kd.
Change ONE gain at a time. Small steps. Always test on a joint you can safely stop.

Tuning Process - Step by Step

Always tune one gain at a time. Never change two at once.

Step 1 Start with Ki=0, Kd=0

- Pure P controller - simplest case, easiest to reason about

Step 2 Increase Kp slowly until it starts to oscillate

- Note the value where oscillation starts - this is your upper bound
- Back off by about 30 percent

Step 3 Add Ki to eliminate steady-state error

- Increase Ki slowly until the arm reliably reaches target
- Watch for windup - if arm slams then recovers slowly, Ki is too high

Step 4 Add Kd to damp any remaining oscillation

- Small values only - Kd amplifies sensor noise if set too high

Rule: if it works and is stable, stop tuning. Perfect is the enemy of good.

Advanced Controllers

Feedforward Control

PID is reactive - it corrects after error already exists.

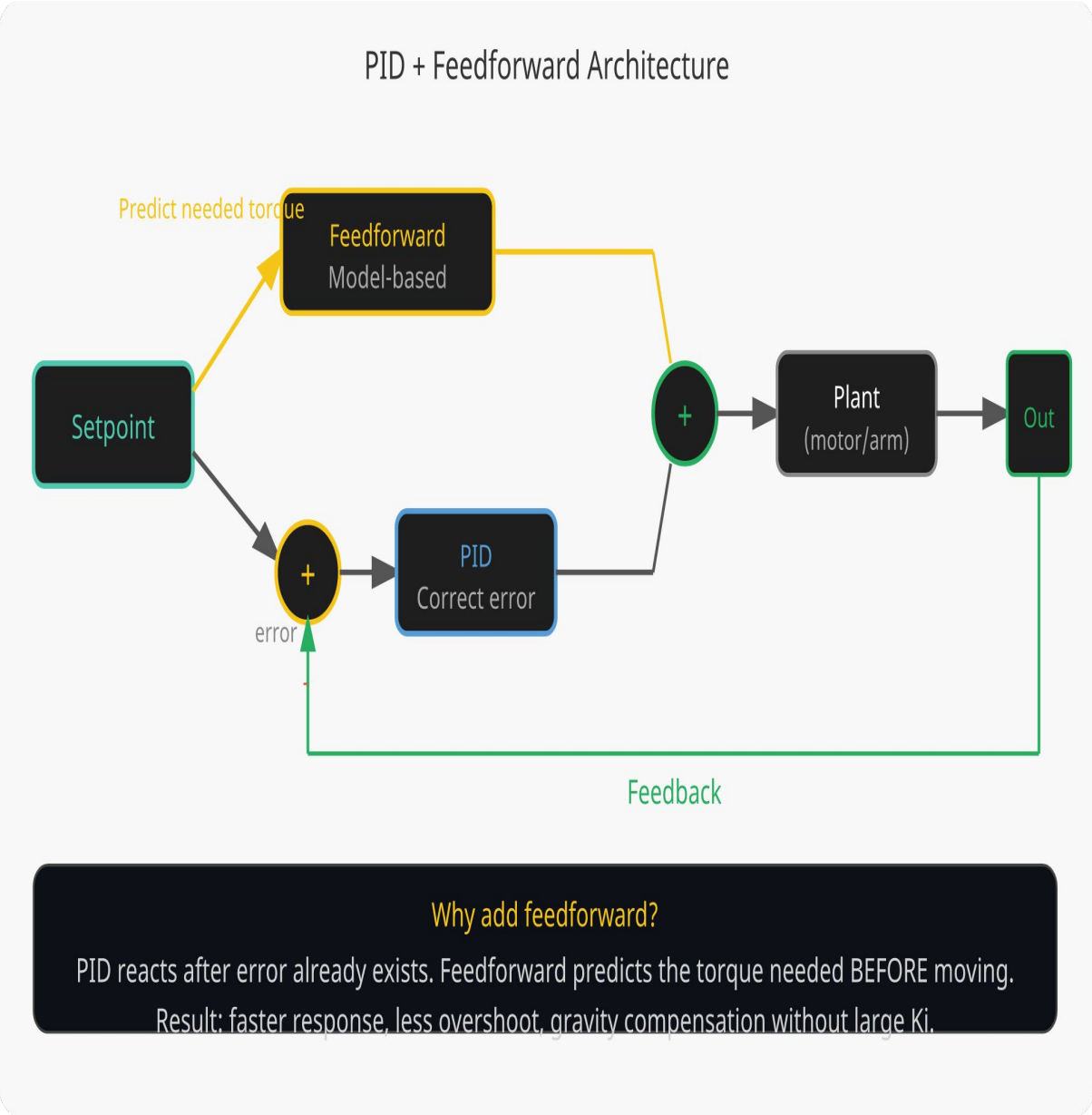
Feedforward is predictive - it estimates the needed output before moving.

How it works:

- Build a model of the plant (motor + arm dynamics)
- Use the model to predict the torque needed for the desired motion
- Add that prediction to the PID output

On the SO-ARM:

- Gravity compensation is a form of feedforward
- Estimate the torque needed to hold position against gravity
- Add it as a constant offset - reduces the steady-state error Ki has to fight



Introduction to LQR

LQR (Linear Quadratic Regulator) is a model-based optimal controller.

How it differs from PID:

- PID has 3 gains you tune by hand with no formal optimality guarantee
- LQR has a cost function you define - it mathematically finds the optimal gains
- LQR accounts for the full system dynamics (mass, inertia, coupling between joints)

The cost function:

- `J = integral( x^T Q x + u^T R u ) minimized over time`
- Q penalizes state error (how much you care about being at target)
- R penalizes control effort (how much you care about motor current)

Why we don't use LQR on the SO-ARM today:

- Requires an accurate dynamic model (mass, inertia of each link) - we don't have that yet
- PID works well enough for our current goals and is far easier to debug

SO-ARM Application

How the SO-ARM Handles Control

Each Feetech STS3215 servo runs its own PID position loop on the motor chip.

Operating modes (the Operating_Mode register)

- 0 Position - you send Goal_Position, the on-chip PID drives the motor there (LeRobot uses this)
- 1 Constant speed - the servo spins at a commanded velocity
- 2 PWM - raw duty cycle, open loop, no on-chip PID
- 3 Step - stepper-style relative moves

There is no torque mode

- The STS3215 cannot take a torque command, so a Python PID cannot drive it directly
- Python talks to the servo at about 30 to 50 Hz over USB, while the chip loop runs much faster

So today we split the work in two:

- Part A: write the PID yourself and run it on a simulated joint where every number is visible
- Part B: tune the real servo PID through P_Coefficient, I_Coefficient, and D_Coefficient

Reading and Setting PID Gains on the SO-ARM

```
# Current LeRobot sets the servo gains through the robot config
# Registers: P_Coefficient=21, D_Coefficient=22, I_Coefficient=23 (D comes before I)

from lerobot.robots.so_follower import SO101Follower, SO101FollowerConfig

PORT = '/dev/ttyACM0'      # yours from lerobot-find-port (COM3, /dev/tty.usbmodem...)
cfg = SO101FollowerConfig(port=PORT, id='my_arm',    # same id you calibrated with
                          position_p_coefficient=16,  # LeRobot defaults
                          position_i_coefficient=0,
                          position_d_coefficient=32)
robot = SO101Follower(cfg)
robot.connect()            # writes these gains to every motor

# Read the gains back from one motor (raw byte values)
for reg in ['P_Coefficient', 'I_Coefficient', 'D_Coefficient']:
    print(reg, robot.bus.read(reg, 'elbow_flex'))

robot.disconnect()         # torque off

# connect() rewrites the gains every time, so a bad value is never permanent.
# Change the numbers in the config, reconnect, and test again.
```

Part A: Your Own PID Loop on a Simulated Joint

```
import csv
import numpy as np

J, B, MGL  = 0.01, 0.02, 0.5     # simulated joint: inertia, friction, gravity
Kp, Ki, Kd = 0.01, 0.0, 0.0      # start P-only, tune from here
setpoint, dt = 45.0, 0.02        # target angle (deg), 50 Hz loop
theta, omega = 0.0, 0.0          # joint angle (deg), joint speed (rad/s)
integral, prev_error = 0.0, 0.0

with open('pid_log.csv', 'w', newline='') as f:
    writer = csv.writer(f)
    writer.writerow(['time', 'setpoint', 'actual', 'error', 'output'])
    for k in range(250):                          # 250 steps = 5 seconds
        error     = setpoint - theta
        integral += error * dt
        deriv     = (error - prev_error) / dt
        output    = Kp*error + Ki*integral + Kd*deriv
        output    = np.clip(output, -2.0, 2.0)      # motor torque limit (N*m)
        # physics: torque in, new angle out
        alpha  = (output - B*omega - MGL*np.cos(np.radians(theta))) / J
        omega += alpha * dt
        theta += np.degrees(omega * dt)
        writer.writerow([round(k*dt, 2), setpoint, theta, error, output])
        prev_error = error
```

Plotting the Response and Diagnosing the Tuning

```
import pandas as pd
import matplotlib.pyplot as plt

df = pd.read_csv('pid_log.csv')        # or one of your step_*.csv files

fig, (ax1, ax2) = plt.subplots(2, 1, figsize=(10, 7), sharex=True)

# Position plot
ax1.plot(df['time'], df['actual'],   label='Actual',   color='steelblue')
ax1.plot(df['time'], df['setpoint'], label='Setpoint', color='green', linestyle='--')
ax1.set_ylabel('Joint Angle (deg)')  ;  ax1.legend()  ;  ax1.grid(True)

# Error and output plot (the real-arm CSV has no output column)
ax2.plot(df['time'], df['error'],  label='Error',  color='tomato')
if 'output' in df:
    ax2.plot(df['time'], df['output'], label='Output', color='gold')
ax2.set_ylabel('Value')  ;  ax2.set_xlabel('Time (s)')
ax2.legend()  ;  ax2.grid(True)

plt.tight_layout()  ;  plt.savefig('pid_response.png')  ;  plt.show()
```

Adding Anti-Windup and a Blocked-Joint Test

```
# Anti-windup: clamp the integral so it can't grow unbounded
INTEGRAL_MAX = 15.0

# Inside the control loop:
integral += error * dt
integral  = np.clip(integral, -INTEGRAL_MAX, INTEGRAL_MAX)    # clamp

# Test it: block the simulated joint at 10 degrees for the first 2 seconds
# (put this right after the physics update)
if k*dt < 2.0 and theta > 10.0:
    theta, omega = 10.0, 0.0

# With Kp=0.05, Ki=0.03, Kd=0.008:
#   No clamp           -> integral winds up, overshoots to about 80 deg on release
#   INTEGRAL_MAX = 15  -> overshoot drops to about 47 deg
#   INTEGRAL_MAX = 10  -> too tight: I can't beat gravity, arm stops near 44 deg
```

Part B: Step Response Test on the Real SO-ARM

```
import time, csv
from lerobot.robots.so_follower import SO101Follower, SO101FollowerConfig
PORT, ARM_ID = '/dev/ttyACM0', 'my_arm'   # your port and calibration id
P, I, D      = 16, 0, 32                  # gain set to test
JOINT, STEP  = 'elbow_flex', 20.0         # move this joint 20 degrees
robot = SO101Follower(SO101FollowerConfig(port=PORT, id=ARM_ID,
    position_p_coefficient=P, position_i_coefficient=I, position_d_coefficient=D))
robot.connect()
start = robot.bus.read('Present_Position', JOINT)
target = start + STEP
try:
    robot.send_action({f'{JOINT}.pos': target})     # the step
    with open(f'step_P{P}_I{I}_D{D}.csv', 'w', newline='') as f:
        w = csv.writer(f)  ;  w.writerow(['time', 'setpoint', 'actual', 'error'])
        t0 = time.time()
        while time.time() - t0 < 3.0:                # log for 3 seconds
            actual = robot.bus.read('Present_Position', JOINT)
            w.writerow([round(time.time()-t0, 3), target, actual, target - actual])
            time.sleep(0.02)                         # about 50 Hz
finally:
    robot.send_action({f'{JOINT}.pos': start})       # always go back
    time.sleep(1.0)  ;  robot.disconnect()           # support the arm: torque goes off
```

Demonstration

Demonstration - Poorly Tuned vs. Properly Tuned

Watch carefully - you will reproduce every step in the hands-on.

What we will show:

- 1. Simulated joint, P-only with Kp too low - the joint sags under gravity
- 2. Raise Kp - faster, but it overshoots and still stops short of 45 degrees
- 3. Add Ki - the steady-state error disappears
- 4. Add Kd - overshoot damps out and settling time drops
- 5. Block the joint with and without the integral clamp and compare the plots
- 6. Real SO-ARM: 20 degree step on elbow_flex with the default gains (P=16, D=32)
- 7. Raise P to 32 and set D to 0 - watch the joint ring, then plot both CSVs together

Pay attention to how much the arm shakes at high P - that is real mechanical stress.

Hands-On

Hands-On - Part A: Tune Your PID in Simulation

Goal: settle within 2 degrees of 45 in under 2 seconds with no oscillation.

1. **Type the simulated-joint script into week7_pid_sim.py**
   - Run it with Kp=0.01, Ki=0, Kd=0, plot it, and explain what you see

2. **Raise Kp until the joint overshoots and rings (try 0.05, then 0.1)**
   - Notice it still settles short of 45 degrees - that is gravity droop

3. **Add Ki (start at 0.02) until the steady-state error is gone**

4. **Add Kd (start at 0.005) until the overshoot is gone**

5. **Add INTEGRAL_MAX and run the blocked-joint test**
   - Save one plot with the clamp and one without

Hands-On - Part B: Tune the Real Servo PID

Goal: compare at least three gain sets on elbow_flex and pick the best one.

1. **Find your port and connect**
   - Run lerobot-find-port, then put your port and calibration id in week7_step_test.py

2. **Baseline: P=16, I=0, D=32 (LeRobot defaults)**
   - Run the 20 degree step test. It saves step_P16_I0_D32.csv

3. **Try P=32, D=0, then P=8, D=32**
   - Watch and listen: which one rings, and which one is sluggish?

4. **Plot all runs on one graph**
   - Load each CSV and plot actual vs. time on the same axes

5. **Pick your best gains and explain the choice using the plot**

Common Mistakes

3 Errors You Will Definitely Hit

Every student hits these. Read this before starting.

Error 1 - Copying old LeRobot code from tutorials

- Imports like lerobot.common.robot_devices and names like P_Gain or Goal_Torque are outdated or never existed
- Use lerobot.robots.so_follower and P_Coefficient / I_Coefficient / D_Coefficient. Check with: pip show lerobot

Error 2 - The arm drops when the script ends

- disconnect() turns torque off, so a raised elbow falls. Support the arm or let it return to rest first
- Wrap the test in try/finally so the joint goes back to start even after Ctrl+C

Error 3 - Integral windup, or a clamp that is too tight

- A blocked joint lets the integral grow, then the release causes a huge overshoot
- Clamp it, but leave room: the I term must be big enough to beat gravity or the arm stops short

Deliverable

Deliverable - Tune a PID in Simulation and on the SO-ARM

Demonstrate all four checkpoints to a lab lead before you start the ROS 2 install:

1 Your own PID running on the simulated joint

- Loop runs at 50 Hz (dt = 0.02). CSV has time, setpoint, actual, error, and output columns.

2 Tuned simulation response

- Settles within 2 degrees of 45 in under 2 seconds with no sustained oscillation. Show before and after plots.

3 Real SO-ARM step response comparison

- At least three gain sets on one plot. Point to the one that rings and the one that is sluggish.

4 Anti-windup and safe stop implemented

- Integral is clamped, the step test uses try/finally, and you can explain why both are necessary.

ROS 2 + Gazebo Setup

What You Are Installing and Why

Starting next week, SI moves from LeRobot scripts to ROS 2 and simulation.

ROS 2 Jazzy

- The Robot Operating System: nodes, topics, services, and launch files that tie a robot together
- Jazzy is the long-term support release built for Ubuntu 24.04

Gazebo Harmonic

- The physics simulator paired with Jazzy - test controllers on a virtual arm before touching hardware

Why Docker with a browser desktop

- One image with Ubuntu 24.04, ROS 2, and Gazebo already installed - same on Windows, Mac, and Linux
- The desktop opens in your browser, so there is no X server or WSL graphics setup
- Simulation needs no USB passthrough, so the Docker problems we hit with the arms do not apply

Heads up: the image is several GB. Start the download first, then read the next slide while it runs.

Install: Pull and Run the ROS 2 Container

```
# 1. Install Docker Desktop (Windows / Mac) or Docker Engine (Linux), open it once
#    https://www.docker.com/products/docker-desktop

# 2. Pull the image: Ubuntu 24.04 + ROS 2 Jazzy + Gazebo Harmonic + browser desktop
docker pull tiryoh/ros2-desktop-vnc:jazzy

# 3. Make a folder for your ROS work (files here survive container restarts)
mkdir ros2_ws

# 4. Create and start the container (run from the folder that holds ros2_ws)
#    Port 6080 = desktop in your browser, 9090 = link to the real arm (next week)
docker run -d --name si_ros2 -p 6080:80 -p 9090:9090 --shm-size=512m \
    -v "$(pwd)/ros2_ws:/home/ubuntu/ros2_ws" \
    tiryoh/ros2-desktop-vnc:jazzy
#    Windows PowerShell: end each line with ` instead of \

# 5. Open the desktop in your browser:   http://127.0.0.1:6080
#    Password if asked: ubuntu

# Next week, start the same container again:
docker start si_ros2
```

Verify Your Install Before You Leave

Open a terminal inside the browser desktop and run both checks:

Check 1 - ROS 2 nodes can talk

- `Terminal 1:  ros2 run demo_nodes_cpp talker`
- `Terminal 2:  ros2 run demo_nodes_py listener`
- The listener should print "I heard: [Hello World: 1]" and keep counting

Check 2 - Gazebo opens

- `ros2 launch ros_gz_sim gz_sim.launch.py gz_args:=shapes.sdf`
- A window with a box, a sphere, and a cylinder appears. Press play.

If Gazebo shows a blank or black window

- Run docker rm -f si_ros2, then rerun step 4 with --security-opt seccomp=unconfined added
- Slow graphics are normal: the container renders without your GPU

Show a lab lead both checks working before you leave.